From Natural Language to Safe Reinforcement Learning with STL

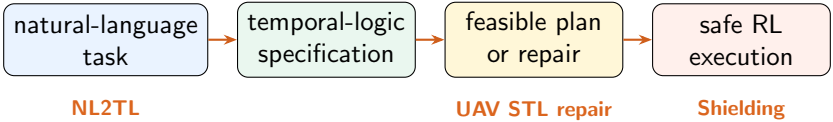
NL2TL (2023) → UAV STL Repair (2026) → Safe RL via Shielding (2017)

Yuhang Zhang

EECS, Washington State University

July 2, 2026

The Shared Story



Reading logic

The three papers cover adjacent but different parts of the same route. NL2TL learns how language becomes temporal logic. The UAV paper checks whether an STL task can produce a feasible trajectory. Shielding enforces a given temporal-logic safety rule during RL.

The missing interface is: **natural-language safety requirements should become executable safety constraints for RL**, not only a translated formula or a one-shot planned trajectory.

Paper 1: NL2TL Learns the Logic Skeleton

Problem

Translate natural-language instructions into temporal logic across domains, despite limited manually labeled NL–TL data.

Core idea

- Hide domain objects and actions as atomic propositions: `prop_1`, `prop_2`, ...
- Fine-tune T5 on lifted NL–STL pairs, so the model mainly learns temporal operators and formula structure.
- Use GPT-3 to help synthesize data and, in application, detect atomic propositions in the original sentence.

Example

"Eventually visit A and always avoid B."

$$\Downarrow$$

$$\mathbf{F}\,\texttt{prop\_1} \;\wedge\; \mathbf{G}\,\neg\texttt{prop\_2}$$

Then `prop_1` and `prop_2` are filled with domain-specific predicates such as regions, objects, or events.

Reported signal

T5-large reaches about 97.5% exact match on lifted data; full NL-to-STL with GPT-3 AP detection is around 95%–97% across tested domains.

NL2TL: What It Gives and What It Does Not

Advantages

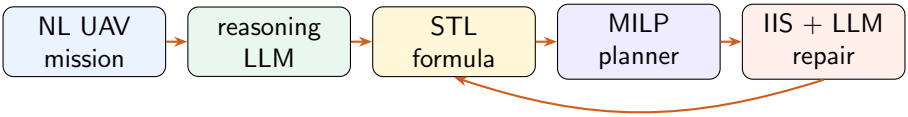
- Separates logical structure from domain grounding.
- Reduces the need for large domain-specific labeled datasets.
- Shows that a fine-tuned sequence model can outperform few-shot GPT-3/GPT-4 end-to-end translation.
- Gives a practical recipe: AP detection plus lifted formula generation.

Limitations for safety control

- Exact formula match is not the same as trajectory-level safety.
- Atomic proposition grounding still depends on domain conventions and language ambiguity.
- The generated formula is not checked for dynamic feasibility.
- There is no RL policy, safety monitor, or runtime intervention mechanism.

NL2TL is best viewed as the **language-to-specification front end**, not as a complete safety-control system.

Paper 2: UAV NL-to-STL with MILP Repair



Key move

The paper does not stop after translation. It encodes STL satisfaction and UAV dynamics into a mixed-integer linear program. If the problem is infeasible, an IIS diagnosis identifies conflicting STL parts; the LLM chooses whether to relax time or predicate requirements, while safety constraints are never relaxed.

Repair example

$$\mathbf{G}_{[0,T]}\neg R_o \wedge \mathbf{F}_{[0,20]}R_g \quad \rightsquigarrow \quad \mathbf{G}_{[0,T]}\neg R_o \wedge \mathbf{F}_{[0,30]}R_g .$$

The obstacle-avoidance rule stays hard; only the goal-reaching deadline is relaxed.

UAV STL Repair: What It Adds and What It Still Assumes

Advantages

- Connects language translation to executable motion planning.
- Uses formal MILP feasibility rather than only text-level correctness.
- Keeps safety constraints non-negotiable during repair.
- Demonstrates the idea in simulation and real low-altitude UAV experiments.

Limitations for our direction

- The downstream executor is a model-based finite-horizon planner, not an RL learner.
- The method assumes known dynamics, known map regions, and MILP-encodable constraints.
- It repairs one planned trajectory, not learning-time exploration over many episodes.
- Safety/task separation and numeric grounding must already be meaningful.

This is the closest paper to the language–STL side, but it answers a planning question rather than a safe-RL question.

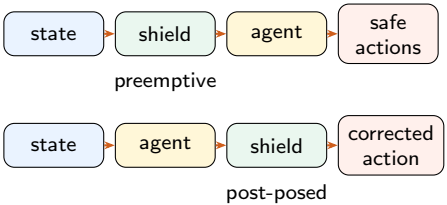
Paper 3: Shielding Makes RL Respect a Given Safety Rule

Problem

RL can learn high-reward behavior while violating safety constraints during training or execution. Reward penalties alone do not provide formal safety guarantees.

Core idea

Given a temporal-logic safety specification φ_s and a conservative finite-state abstraction of the environment, synthesize a shield that sits between the agent and the environment.



Guarantee target: **correctness** and **minimum interference**; the shield blocks only actions that could force a future violation.

Shielded RL: What It Gives and What It Does Not

Advantages

- Safety is enforced at runtime, including during learning.
- The shield is mostly independent of the internal RL algorithm.
- It reasons ahead: an action can be blocked before the violation becomes immediate.
- It separates safety correctness from reward optimization.

Limitations for language-driven safety

- The safety specification φ_s is assumed to be given by an expert.
- A conservative environment abstraction is required.
- Rich numeric STL constraints need grounding and abstraction before shielding.
- It does not translate, repair, or disambiguate natural-language requirements.

Shielding is the strongest safety-execution piece, but it starts after the specification already exists.

Natural Gap: NL Safety Constraints for RL

Focused problem

Input: an RL environment M and a language task $L = (L_{\text{goal}}, L_{\text{safe}})$. Translate L_{safe} into grounded STL and test whether it makes RL measurably safer:

$$\max_{\pi} \ \mathbb{E}_{\pi}\left[\sum_{t=0}^{H} \gamma^t r_t\right] \quad \text{s.t.} \quad \Pr_{\tau\sim\pi}[\tau \models \varphi_{\text{safe}}] \geq 1 - \delta.$$

Why this is not exactly prior work

- NL2TL translates formulas, but does not enforce them.
- UAV repair enforces STL through MILP planning, not RL learning.
- Shielding enforces RL safety, but assumes a formal rule.

Key difficulties

- Ground vague language into predicates and time windows.
- Separate hard safety from soft task preference.
- Evaluate trajectory violations, not only exact match.
- Handle infeasibility and abstraction error.